

EDS Lab Assignments

Practical 4

Name: Swamini Bhagwat

PRN: 202501100023

4.1.1 Pandas – Series Creation and Manipulation

Problem Statement:

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Code:

```
import pandas as pd

# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))

# Create a Pandas series from the list of numbers
series = pd.Series(numbers)

# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()

# Display the mean of even and odd numbers with labels
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
print("Mean of even and odd numbers:")
print(grouped)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a problem statement is visible: "Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations." The main editor shows the following Python code:

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8
9 # Grouping by even and odd numbers and calculating the mean
10 grouped = series.groupby(series % 2).mean()
11
12 # Display the mean of even and odd numbers with labels
13 grouped.index = ["Even" if is_even else "Odd" for is_even in grouped.index]
14 print("Mean of even and odd numbers:")
15 print(grouped)
```

The output pane shows the results of two test cases. Test case 1 has an expected output of "Mean of even and odd numbers: 1.5 4.5" and an actual output of "Mean of even and odd numbers: 1.5 4.5". Test case 2 has an expected output of "Mean of even and odd numbers: 6.833333 12.166667" and an actual output of "Mean of even and odd numbers: 6.833333 12.166667". The program passed all three test cases.

4.1.2 Dictionary to Dataframe

Problem Statement:

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Code:

```
import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new
Name = input("New name: ")
Age = int(input("New age: "))
new = {'Name':Name, 'Age':Age}
df.loc[len(df)] = new
# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
index = int(input("Index of row to modify: "))
age = int(input("New age: "))
df.at[index, 'Age']=age
# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
delt = int(input("Index of row to delete: "))
```

```
df = df.drop(delt).reset_index(drop=True)
# Display the DataFrame after deleting a row
print("After deleting a row:")
print(df)
```

```
# Adding a new column
gen = input("Enter genders separated by space: ").split()
df['Gender']=gen
# Display the DataFrame after adding a new column
print("After adding a new column:")
print(df)
```

```
# Modifying a column
df['Name']=df['Name'].str.upper()
# Display the DataFrame after modifying a column
print("After modifying a column:")
print(df)
```

```
# Deleting a column
df = df.drop(columns = ['Age'])\
# Display the DataFrame after deleting a column
print("After deleting a column:")
print(df)
```

Output:

The screenshot shows the CODETANTRA IDE interface. On the left, a sidebar lists various operations: Create the DataFrame, Add a new row, Modify a row, Delete a row, Add a new column, Modify a column, and Delete a column. The main editor displays a Python script that performs the following steps:

- Import pandas as pd
- Provide a dictionary of lists: `data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]}`
- Convert the dictionary to a DataFrame: `df = pd.DataFrame(data)`
- Display the original DataFrame: `print("Original DataFrame:")`
- Adding a new row: `new_name = input("New name: ")`, `new_age = int(input("New age: "))`, `df.loc[len(df)] = [new_name, new_age]`
- Display the DataFrame after adding a new row: `print("After adding a row:")`
- Modifying a row: `row_to_modify = int(input("Index of row to modify: "))`, `new_age_value = int(input("New age: "))`, `df.at[row_to_modify, "Age"] = new_age_value`
- Display the DataFrame after modifying a row: `print("After modifying a row:")`
- Deleting a row: `row_to_delete = int(input("Index of row to delete: "))`, `df = df.drop(index = row_to_delete).reset_index(drop=True)`
- Display the DataFrame after deleting a row: `print("After deleting a row:")`

The output of the script is shown in the terminal at the bottom, displaying the DataFrame at each step of the process.

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.
- Display the DataFrame after deleting the column.

Sample Test Cases

```

23
24 # Modifying a row
25 row_to_modify = int(input("Index of row to modify: "))
26 new_age_value = int(input("New age: "))
27 df.at[row_to_modify, "Age"] = new_age_value
28
29
30 # Display the DataFrame after modifying a row
31 print("After modifying a row:")
32 print(df)
33
34 # Deleting a row
35 row_to_delete = int(input("Index of row to delete: "))
36 df = df.drop(index=row_to_delete).reset_index(drop=True)
37 # Display the DataFrame after deleting a row
38 print("After deleting a row:")
39 print(df)
40
41
42 # Adding a new column
43 genders = input("Enter genders separated by space: ").split()
44 df["Gender"] = genders
45
46 # Display the DataFrame after adding a new column
47 print("After adding a new column:")
48 print(df)
49
50 # Modifying a column
51 df["Name"] = df["Name"].str.upper()
52
53 # Display the DataFrame after modifying a column
54 print("After modifying a column:")
55 print(df)
56
57 # Deleting a column
58 df = df.drop(columns=["Age"])
59 # Display the DataFrame after deleting a column
60 print("After deleting a column:")
61 print(df)

```

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column Gender with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the Age column.
- Display the DataFrame after deleting the column.

Sample Test Cases

Average time: 0.052 s
Maximum time: 0.062 s
1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed

Test case 1

Expanded output	Actual output
Original DataFrame:	Original DataFrame:
..... Name Age	 Name Age
0 Alice 25	0 Alice 25
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
New row: (Name, Age)	New row: (Name, Age)
New age: 27	New age: 27
After adding a row:	After adding a row:
..... Name Age	 Name Age
0 Alice 25	0 Alice 25
1 Bob 30	1 Bob 30
2 Charlie 35	2 Charlie 35
3 Susan 29	3 Susan 29
Index of row to modify: 1	Index of row to modify: 1
New age: 27	New age: 27
After modifying a row:	After modifying a row:
..... Name Age	 Name Age
0 Alice 25	0 Alice 25
1 Bob 27	1 Bob 27
2 Charlie 35	2 Charlie 35
3 Susan 29	3 Susan 29
Index of row to delete: 1	Index of row to delete: 1
After deleting a row:	After deleting a row:
..... Name Age	 Name Age
0 Alice 25	0 Alice 25
1 Bob 27	1 Bob 27
2 Charlie 35	2 Charlie 35
3 Susan 29	3 Susan 29
Enter genders separated by space: (F, M)	Enter genders separated by space: (F, M)
After adding a new column:	After adding a new column:
..... Name Age Gender	 Name Age Gender
0 Alice 25 F	0 Alice 25 F
1 Bob 27 M	1 Bob 27 M
2 Charlie 35 F	2 Charlie 35 F
3 Susan 29 M	3 Susan 29 M

4.1.3 Student Information

Problem Statement:

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Code:

```

import pandas as pd
# Read the text file into a DataFrame

```

```

file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
# write your code here..
print("First five rows:")
print(data.head())
average_age = round(data["Age"].mean(), 2)
print(f"Average age: {average_age}")
filtered_students = data[data["Grade"] <= "B"]
print("Students with a grade up to B")
print(filtered_students)

```

Output:

The screenshot shows a coding environment with a Python script and its execution results. The script reads a CSV file, calculates the average age, and filters students by grade. The output shows the first five rows of the data and the filtered students.

Sample Test Cases:

- Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:
 - Display the first five rows of the data frame.
 - Calculate the average age of the students (limit the average age up to 2 decimal places).
 - Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note: Refer to the displayed test cases for better understanding.

Test Case 1: 1 out of 1 shown test case(s) passed. 1 out of 1 hidden test case(s) passed.

Expected output:

```

studentdata.csv
First five rows:
  Name Age Grade
0 John  20    A
1 Allen 22    B
2 Emma  24    A
3 Mike  23    C
4 Sony  21    B
Average age: 23.20
Students with a grade up to B:
  Name Age Grade
0 John  20    A
1 Allen 22    B
2 Emma  24    A
4 Sony  21    B
5 Anila 26    A

```

Actual output:

```

studentdata.txt
First five rows:
  Name Age Grade
0 John  20    A
1 Allen 22    B
2 Emma  24    A
3 Mike  23    C
4 Sony  21    B
Average age: 23.20
Students with a grade up to B:
  Name Age Grade
0 John  20    A
1 Allen 22    B
2 Emma  24    A
4 Sony  21    B
5 Anila 26    A

```

4.2.1 Month with the Highest Total Sales

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Code:

```

import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)

```

```
df['Date'] = pd.to_datetime(df['Date'])
df['Total_Sales_Amount'] = df['Quantity'] * df['Price']
Monthly_Sales = df.groupby(df['Date'].dt.strftime('%Y-%m'))['Total_Sales_Amount'].sum()
```

```
# Find the month with the highest total sales
best_month = Monthly_Sales.idxmax()
highest_sales = Monthly_Sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

Output:

The screenshot shows a CodeTANTRA IDE interface. On the left, there's a sidebar with a file explorer and a console. The main editor displays a Python script for finding the best month with the highest total sales. The script uses pandas to read a CSV file, calculate total sales by month, and find the month with the highest sales. The output shows the best month as 2023-05 and the total sales as \$1218.00.

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # Find the month with the highest total sales
10 df['Date'] = pd.to_datetime(df['Date'])
11 df['Month'] = df['Date'].dt.to_period("M")
12 df['Total Sales'] = df['Quantity'] * df['Price']
13 monthly_sales = df.groupby("Month")["Total Sales"].sum()
14
15 best_month = monthly_sales.idxmax()
16 highest_sales = monthly_sales.max()
17
18 print(f"Best month: {best_month}")
19 print(f"Total sales: ${highest_sales:.2f}")
20
```

The output section shows the following results:

Test Case	Expected Output	Actual Output
Test case 1	Best month: 2023-05 Total sales: \$1218.00	Best month: 2023-05 Total sales: \$1218.00

4.2.2 Best Selling Product

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Code:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

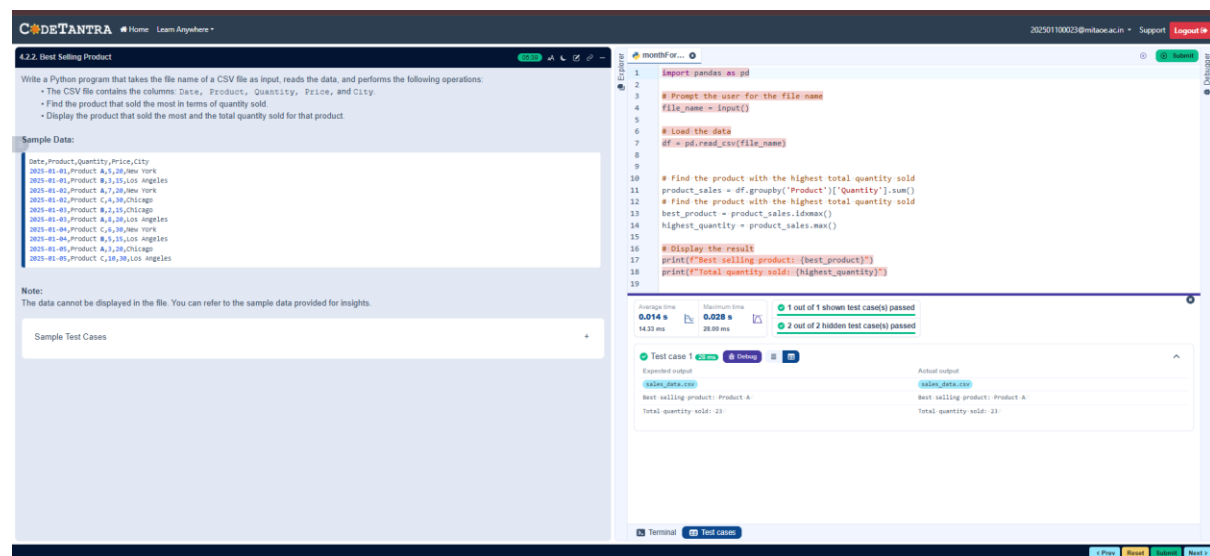
```
# Load the data
```

```

df = pd.read_csv(file_name)
product_sales = df.groupby("Product")["Quantity"].sum()
# Find the product with the highest total quantity sold
best_product = product_sales.idxmax()
highest_quantity = product_sales.max()
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")

```

Output:



4.2.3 City that Sold the Most Products

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Code:

```

import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

```

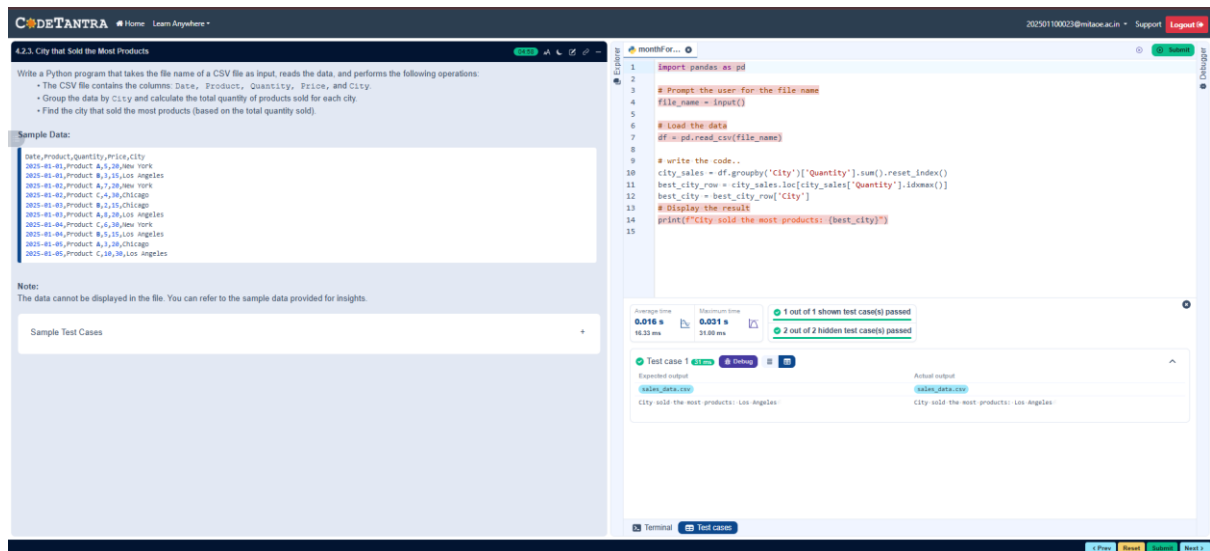

write the code..

```
city_sales = df.groupby("City")["Quantity"].sum()
best_city = city_sales.idxmax()
```

Display the result

```
print(f"City sold the most products: {best_city}")
```

Output:



4.2.4 Most Frequently Sold Product Pairs

Problem Statement:

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Code:

```
import pandas as pd
```

```
from itertools import combinations
```

```
from collections import Counter
```

Prompt user to input the file name

```
file_name = input()
```

Read data from the specified CSV file

```
df = pd.read_csv(file_name)
```

write the code

```
grouped = df.groupby("Date")["Product"].apply(list)
```

```
pairs_counter = Counter()
```

for products in grouped:

```
pairs = combinations(sorted(products), 2)
```

```
pairs_counter.update(pairs)
```

```
max_frequency = max(pairs_counter.values())
```

```
most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq ==  
max_frequency]
```

Output the most frequent product pairs

for(product1, product2), frequency in most_common_pairs:

```
print(f"{product1} and {product2}: {frequency} times")
```

Output:

4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pairs that were sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	10	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	18	Chicago
2025-01-03	Product B	6	12	Chicago
2025-01-03	Product A	8	25	Los Angeles
2025-01-04	Product C	6	16	New York
2025-01-04	Product B	5	13	Los Angeles
2025-01-05	Product A	1	10	Chicago
2025-01-05	Product C	1	18	Los Angeles

Explanation:

Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appear in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B: (2 times)
- Product A and Product C: (2 times)

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12 grouped = df.groupby("Date")["Product"].apply(list)
13 pairs_counter = Counter()
14 for products in grouped:
15     pairs = combinations(sorted(products), 2)
16     pairs_counter.update(pairs)
17 max_frequency = max(pairs_counter.values())
18 most_common_pairs = [(pair, freq) for pair, freq in pairs_counter.items() if freq == max_frequency]
19
20 # Output the most frequent product pairs
21 for (product1, product2), frequency in most_common_pairs:
22     print(f"{product1} and {product2}: {frequency} times")
23
```

Test Results:

- 1 out of 1 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test Case	Expected Output	Actual Output
Product A and Product B: 2 times	Product A and Product B: 2 times	Product A and Product B: 2 times
Product A and Product C: 2 times	Product A and Product C: 2 times	Product A and Product C: 2 times

4.2.5 Titanic Dataset Analysis and Data Cleaning

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
print(data.head())

# 1. Display the first 5 rows of the dataset
print(data.tail())

# 2. Display the last 5 rows of the dataset
print(data.shape)

# 3. Get the shape of the dataset
print(data.info())

# 4. Get a summary of the dataset (info)
print(data.describe())
```

5. Get basic statistics of the dataset

```
print(data.isnull().sum())
```

6. Check for missing value

```
median_age = data['Age'].median()
```

```
data['Age'].fillna(median_age, inplace = True)
```

7. Fill missing values in the 'Age' column with the median age

```
mode_embarked = data['Embarked'].mode()[0]
```

```
data['Embarked'].fillna(mode_embarked, inplace = True)
```

8. Fill missing values in the 'Embarked' column with the mode

```
data.drop('Cabin', axis=1, inplace = True)
```

9. Drop the 'Cabin' column due to many missing values

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

Output:

4.2.5. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Mr. Owen Harris	male	22	1	0	A/5 21171,72,5	5.12		S
2	1	1	Mr. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599,71,2833,C85,C	53.1		S
3	1	3	Mr. James (Mrs. James) (Miss. Lavinia)	female	26	0	0	STON/O2. 3101282,7,325,5	21.01		S
4	1	1	Mr. Jacques Heath (Lily May Peel)	female	35	1	0	113803,51,1,C123,5	51.66		S
5	0	3	Mr. William Henry	male	30	0	0	373450,8,95,5	16.99		S
6	0	3	Mr. James	male	0	0	0	330877,8,4503,0	0.00		S
7	0	1	Mr. Timothy J.	male	34	0	0	17463,51,8625,846,5	51.66		S
8	0	3	Mr. Leonard	male	23	1	0	349909,21,075,5	16.99		S
9	1	3	Mr. Oscar W (Elizabeth Vilhelmina Berg)	female	27	0	1	347742,11,1338,5	33.69		S
10	1	3	Mr. Nicholas (Adele Schenck)	female	18	1	0	337758,10,0798,5	21.01		S

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1, 0, 3, Mr. Owen Harris, male, 22, 1, 0, A/5 21171,72,5, 5.12, , S
2, 1, 1, Mr. John Bradley (Florence Briggs Thayer), female, 38, 1, 0, PC 17599,71,2833,C85,C, 53.1, , S
3, 1, 3, Mr. James (Mrs. James) (Miss. Lavinia), female, 26, 0, 0, STON/O2. 3101282,7,325,5, 21.01, , S
4, 1, 1, Mr. Jacques Heath (Lily May Peel), female, 35, 1, 0, 113803,51,1,C123,5, 51.66, , S
5, 0, 3, Mr. William Henry, male, 30, 0, 0, 373450,8,95,5, 16.99, , S
6, 0, 3, Mr. James, male, 0, 0, 0, 330877,8,4503,0, 0.00, , S
7, 0, 1, Mr. Timothy J., male, 34, 0, 0, 17463,51,8625,846,5, 51.66, , S
8, 0, 3, Mr. Leonard, male, 23, 1, 0, 349909,21,075,5, 16.99, , S
9, 1, 3, Mr. Oscar W (Elizabeth Vilhelmina Berg), female, 27, 0, 1, 347742,11,1338,5, 33.69, , S
10, 1, 3, Mr. Nicholas (Adele Schenck), female, 18, 1, 0, 337758,10,0798,5, 21.01, , S
```

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 # 1. Display the first 5 rows of the dataset
7 print(data.head())
8
9 # 2. Display the last 5 rows of the dataset
10 print(data.tail())
11
12 # 3. Get the shape of the dataset
13 print(data.shape)
14
15 # 4. Get a summary of the dataset (info)
16 print(data.info())
17
18 # 5. Get basic statistics of the dataset
19 print(data.describe())
20
21 # 6. Check for missing values
22 print(data.isnull().sum())
23
24 # 7. Fill missing values in the 'Age' column with the median age
25 data['Age'].fillna(data['Age'].median(), inplace=True)
26
27 # 8. Fill missing values in the 'Embarked' column with the mode
28 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
29
30 # 9. Drop the 'Cabin' column due to many missing values
31 data.drop('Cabin', axis=1, inplace=True)
32
33 # 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
34 data['FamilySize'] = data['SibSp'] + data['Parch']
35
```

42.5. Titanic: Dataset Analysis and Data Cleaning

0.091 s
91.00 ms

0.091 s
91.00 ms

1 out of 1 shown test case(s) passed

Test Case 1

Debug

Test

Expected output

Actual output

PassengerId Survived Pclass Fare Cabin Embarked

0 1 3 7.2500 NaN S

1 0 1 71.2833 C85 C

2 1 3 7.2500 NaN S

3 1 3 53.1000 C123 S

4 0 3 51.0000 NaN S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked

886 1 0 12.00 NaN S

887 0 1 30.00 042 S

888 0 3 23.45 NaN S

889 1 1 30.00 C140 C

890 0 3 7.75 NaN Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

column non-null count dtype

PassengerId 891 non-null int64

Survived 891 non-null int64

Pclass 891 non-null int64

Name 891 non-null object

Sex 891 non-null object

Age 714 non-null float64

SibSp 891 non-null int64

Parch 891 non-null int64

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

column non-null count dtype

PassengerId 891 non-null int64

Survived 891 non-null int64

Pclass 891 non-null int64

Name 891 non-null object

Sex 891 non-null object

Age 714 non-null float64

SibSp 891 non-null int64

Parch 891 non-null int64

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Brands, Mr. Owen Harris",male,20,1,0,A/5 21171,7.25,S
2,1,1,"Cummings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikinen, Miss. Laina",female,26,0,8,STON/O2. 3101282,7.925,S
4,1,1,"Hartley, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373459,8.85,S
6,0,3,"Moran, Mr. James",male,0,0,330877,0.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,Q46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Ellenest Vilhelmine Berg)",female,27,0,2,347742,11.1333,S
10,1,2,"Meyer, Mrs. Nicholas (Adelce Acene)",female,14,1,0,237736,30.4708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

42.5. Titanic: Dataset Analysis and Data Cleaning

0.091 s
91.00 ms

0.091 s
91.00 ms

1 out of 1 shown test case(s) passed

Test Case 1

Debug

Test

Expected output

Actual output

PassengerId Survived Pclass Fare Cabin Embarked

0 1 3 7.2500 NaN S

1 0 1 71.2833 C85 C

2 1 3 7.2500 NaN S

3 1 3 53.1000 C123 S

4 0 3 51.0000 NaN S

[5 rows x 12 columns]

PassengerId Survived Pclass Fare Cabin Embarked

886 1 0 12.00 NaN S

887 0 1 30.00 042 S

888 0 3 23.45 NaN S

889 1 1 30.00 C140 C

890 0 3 7.75 NaN Q

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

column non-null count dtype

PassengerId 891 non-null int64

Survived 891 non-null int64

Pclass 891 non-null int64

Name 891 non-null object

Sex 891 non-null object

Age 714 non-null float64

SibSp 891 non-null int64

Parch 891 non-null int64

[5 rows x 12 columns]

(891, 12)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

column non-null count dtype

PassengerId 891 non-null int64

Survived 891 non-null int64

Pclass 891 non-null int64

Name 891 non-null object

Sex 891 non-null object

Age 714 non-null float64

SibSp 891 non-null int64

Parch 891 non-null int64

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Brands, Mr. Owen Harris",male,20,1,0,A/5 21171,7.25,S
2,1,1,"Cummings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikinen, Miss. Laina",female,26,0,8,STON/O2. 3101282,7.925,S
4,1,1,"Hartley, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373459,8.85,S
6,0,3,"Moran, Mr. James",male,0,0,330877,0.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,Q46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Ellenest Vilhelmine Berg)",female,27,0,2,347742,11.1333,S
10,1,2,"Meyer, Mrs. Nicholas (Adelce Acene)",female,14,1,0,237736,30.4708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

4.2.6 Titanic Dataset Analysis and Data Cleaning-2

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.

9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['Alone'] = np.where(data['FamilySize'] == 0, 1, 0)

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns = ['Embarked'], drop_first = True)

# 4. Get the mean age of passengers
mean_age = data['Age'].mean()
print(mean_age)

# 5. Get the median fare of passengers
median_fare = data['Fare'].median()
print(median_fare)

# 6. Get the number of passengers by class
passengers_by_class = data['Pclass'].value_counts()
print(passengers_by_class)

# 7. Get the number of passengers by gender
passengers_by_gender = data['Sex'].value_counts().sort_index()
print(passengers_by_gender)

# 8. Get the number of passengers by survival status
passengers_by_survival = data['Survived'].value_counts().sort_index()
print(passengers_by_survival)
```

```
# 10. Calculate the survival rate by gender
survival_rate_by_gender = data.groupby('Sex')['Survived'].mean()
print(survival_rate_by_gender)
```

Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone ('FamilySize' = 0), otherwise 0.

2. Convert the 'Sex' column to numeric values (male: 0, female: 1).

3. One-hot encode the 'Embarked' column, dropping the first category.

4. Get the mean age of passengers.

5. Get the median fare of passengers.

6. Get the number of passengers by class.

7. Get the number of passengers by gender.

8. Get the number of passengers by survival status.

9. Calculate the survival rate of passengers.

10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

Passen gerId	Survive d	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embar ked
Sample Data:											
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171,726	5.12		
2	1	1	Colley, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	M/C 17999,71,2833,085,C	7.25		
3	1	3	Hickman, Miss. Laina	female	26	0	8	STON/O 3101292,7325,S	5.65		
4	1	1	Futrelle, Mr. Jacques Heath (Lily May Peel)	female	35	1	0	113889,53,1,C123,S	7.25		
5	0	3	Allan, Mr. William Henry	male	29	0	0	37460,9,405,S	5.41		
6	0	3	Norman, Mr. James	male	0	0	0	330807,8,4563,Q	21.01		
7	0	1	O'Carroll, Mr. Timothy J	male	54	0	0	27461,51,8023,446,S	53.1		
8	0	3	Kilian, Master. Gustav Leonard	male	21	1	1	349089,11,675,S	16.71		
9	1	3	Joneson, Mrs. Oscar W (Ellenath Vilhelmina Berg)	female	27	0	2	347742,11,1133,S	31.27		
10	1	2	Messer, Mrs. Nicholas (Adèle Achen)	female	34	1	0	237726,30,0786,C	16.71		

Note: Refer to the visible test case for better reference.

Sample Test Cases

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 # 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
9 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
12 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
13
14 # 3. One-hot encode the 'Embarked' column
15 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
16
17 # 4. Get the mean age of passengers
18 print(data['Age'].mean())
19
20 # 5. Get the median fare of passengers
21 print(data['Fare'].median())
22
23 # 6. Get the number of passengers by class
24 print(data['Pclass'].value_counts())
25
26 # 7. Get the number of passengers by gender
27 print(data['Sex'].value_counts())
28
29 # 8. Get the number of passengers by survival status
30 print(data['Survived'].value_counts())
31
32 # 9. Calculate the survival rate
33 print(data['Survived'].mean())
34
35 # 10. Calculate the survival rate by gender
36 print(data.groupby('Sex')['Survived'].mean())
37
```

1.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
- Convert the 'Sex' column to numeric values (male: 0, female: 1).
- One-hot encode the 'Embarked' column, dropping the first category.
- Get the mean age of passengers.
- Get the median fare of passengers.
- Get the number of passengers by class.
- Get the number of passengers by gender.
- Get the number of passengers by survival status.
- Calculate the survival rate of passengers.
- Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Henry", male, 22,1,0,0, 71213721,7.25,S
2,1,3,"Cuthbert, Mrs. Jane Bradley (Florence Briggs Thayer)", female, 38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina", female, 26,0,0,STOVJZ, 51.00,0
4,1,1,"Futrelle, Mr. Jacques Heath (Lily May Peel)", female, 35,1,0,113803,53.1,C123,S
5,0,3,"Miller, Mr. William Henry", male, 35,0,0,37536,8.65,S
6,0,3,"Moran, Mr. James", male, 0,0,13807,0.4533,Q
7,0,1,"McCarthy, Mr. Timothy J", male, 34,0,0,17461,51.8631,Q4,S
8,0,3,"Palsson, Master. Gosta Leonard", male, 2,1,1,340969,21.075,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmine Berg)", female, 27,0,2,347742,11.1393,S
10,1,2,"Nasser, Mrs. Nicholas (Adelie Achen)", female, 54,1,0,237736,30.0708,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

Test Case 1 Run Debug

Average time: **0.037 s**
17.60 ms

Maximum time: **0.037 s**
37.60 ms

1 out of 1 shown test case(s) passed

Actual output

```
29.49911764796882
14.4542
3 --- 493
1 --- 216
2 --- 184
Name: Pclass, dtype: int64
0 --- 877
1 --- 214
Name: Sex, dtype: int64
0 --- 549
1 --- 342
Name: Survived, dtype: int64
0.3032830303030303
Sex
0 --- 0.183968
1 --- 0.742038
Name: Survived, dtype: float64
```

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

Code:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class
print(data.groupby('Pclass')['Survived'].mean())

# 2. Calculate the survival rate by embarked location
print(data.groupby('Embarked_S')['Survived'].mean())

# 3. Calculate the survival rate by family size
print(data.groupby('FamilySize')['Survived'].mean())

# 4. Calculate the survival rate by being alone
print(data.groupby('IsAlone')['Survived'].mean())

# 5. Get the average fare by class
print(data.groupby('Pclass')['Fare'].mean())

# 6. Get the average age by class
```



```
print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Output:

The screenshot shows a Jupyter Notebook titled "4.2.7. Titanic Dataset Analysis and Data Cleaning - 3". The notebook contains a list of 10 tasks and a corresponding Python code block. The code uses pandas to load the Titanic dataset, calculate survival rates by class, embarked location, family size, and alone status, and calculate average age and fare by survival status. It also uses value_counts to get the number of survivors and non-survivors by class.

The output of the code is displayed in the notebook cells. The first cell shows the survival rate by class, embarked location, family size, and alone status. The second cell shows the average age and fare by survival status. The third cell shows the number of survivors and non-survivors by class.

Passen gerId	Survive d	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embark ed
1	0	3	Brand, Mr. Owen Harris	male	22	1	0	A/5 21171,72	5		S
2	1	1	Cunings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599,71	2833	C85,C	C
3	1	3	Hickelken, Miss. Laina	female	26	0	0	STON/O2. 3101282,7	925		S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113380,53	1	C123,S	S
5	0	3	Allen, Mr. William Henry	male	35	0	0	37450,8	85		S
6	0	3	Noran, Mr. James	male	40	0	0	10077,6	458		S
7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463,51	8625	E46,S	S
8	0	3	Paizon, Master. Gosta Leonard	male	2	3	1	349909,21	875		S
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	1	34742,11	1333		S
10	1	2	Hesser, Mrs. Nicholas (Adèle Achon)	female	14	1	0	237736,38	4768		C

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

The screenshot shows the same Jupyter Notebook as above, but with the output of the code displayed in the notebook cells. The first cell shows the survival rate by class, embarked location, family size, and alone status. The second cell shows the average age and fare by survival status. The third cell shows the number of survivors and non-survivors by class.

The output of the code is displayed in the notebook cells. The first cell shows the survival rate by class, embarked location, family size, and alone status. The second cell shows the average age and fare by survival status. The third cell shows the number of survivors and non-survivors by class.

Passen gerId	Survive d	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embark ed
1	0	3	Brand, Mr. Owen Harris	male	22	1	0	A/5 21171,72	5		S
2	1	1	Cunings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599,71	2833	C85,C	C
3	1	3	Hickelken, Miss. Laina	female	26	0	0	STON/O2. 3101282,7	925		S
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113380,53	1	C123,S	S
5	0	3	Allen, Mr. William Henry	male	35	0	0	37450,8	85		S
6	0	3	Noran, Mr. James	male	40	0	0	10077,6	458		S
7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463,51	8625	E46,S	S
8	0	3	Paizon, Master. Gosta Leonard	male	2	3	1	349909,21	875		S
9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	1	34742,11	1333		S
10	1	2	Hesser, Mrs. Nicholas (Adèle Achon)	female	14	1	0	237736,38	4768		C

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

4.2.7: Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Calculate the survival rate by class.
- Calculate the survival rate by embarkation location (Embarked_S).
- Calculate the survival rate by family size (FamilySize).
- Calculate the survival rate by being alone (IsAlone).
- Get the average fare by passenger class (Pclass).
- Get the average age by passenger class (Pclass).
- Get the average age by survival status (Survived).
- Get the average fare by survival status (Survived).
- Get the number of survivors by class (Pclass).
- Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris", male, 22, 1, 0, 0, 5, 2171, 7.25, S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)", female, 38, 1, 0, 0, PC 17598, 71.2833, C85, C
3,1,3,"Heikkinen, Miss. Laina", female, 26, 0, 0, 5, 0, 0, 3101282, 7.925, S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)", female, 35, 1, 0, 1, 13803, 53.1, C123, S
5,0,3,"Allen, Mr. William Henry", male, 29, 0, 0, 0, 0, 0, 31440, 8.45, S
6,0,3,"Moran, Mr. James", male, 0, 0, 330877, 0, 453, 0
7,0,1,"McCarthy, Mr. Timothy J", male, 34, 0, 0, 1, 0, 0, 51, 51.0, S
8,0,3,"Palsson, Master. Gosta Leonard", male, 2, 1, 1, 1, 0, 0, 9999, 21.075, S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)", female, 27, 0, 2, 0, 347742, 11.1333, S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)", female, 14, 1, 0, 0, 237736, 30.4708, C

```

Note: Refer to the visible test case for better reference.

Sample Test Cases

0.052 s

52.00 ms

0.052 s

52.00 ms

1 out of 1 shown test case(s) passed

Debugger

```

7: Name: Survived, dtype: float64
10: Name: Survived, dtype: float64
IsAlone:
0: 0.500000
1: 0.500000
Name: Survived, dtype: float64
Pclass:
1: 0.4154687
2: 0.462183
3: 0.475550
Name: Fare, dtype: float64
Pclass:
1: 38.233441
2: 29.877638
3: 25.148628
Name: Age, dtype: float64
Survived:
0: 26.426179
1: 25.343090
Name: Age, dtype: float64
Survived:
0: 22.117887
1: 48.395488
Name: Fare, dtype: float64
1: 136
3: 119
2: 87
Name: Pclass, dtype: int64
3: 372
2: 97
3: 88
Name: Pclass, dtype: int64

```

Prev

Reset

Submit

Next >

4.2.8 Titanic Dataset Analysis and Data Cleaning-4

Problem Statement:

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- Get the number of survivors by gender (Sex).
- Get the number of non-survivors by gender (Sex).
- Get the number of survivors by embarkation location (Embarked_S).
- Get the number of non-survivors by embarkation location (Embarked_S).
- Calculate the percentage of children (Age < 18) who survived.
- Calculate the percentage of adults (Age >= 18) who survived.
- Get the median age of survivors.
- Get the median age of non-survivors.
- Get the median fare of survivors.
- Get the median fare of non-survivors.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Get the number of survivors by gender
```

```
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
```

```
print(survivors_by_gender)
```

2. Get the number of non-survivors by gender

```
non_survivors_by_gender = data[data['Survived'] == 0]['Sex'].value_counts()
print(non_survivors_by_gender)
```

3. Get the number of survivors by embarked location

```
survivors_by_embarked_s = data[data['Survived'] == 1]['Embarked_S'].value_counts()
print(survivors_by_embarked_s)
```

4. Get the number of non-survivors by embarked location

```
non_survivors_by_embarked_s = data[data['Survived'] == 0]['Embarked_S'].value_counts()
print(non_survivors_by_embarked_s)
```

5. Calculate the percentage of children (Age < 18) who survived

```
children = data[data['Age'] < 18]
children_survival_rate = children['Survived'].mean()
print(children_survival_rate)
```

6. Calculate the percentage of adults (Age >= 18) who survived

```
adults = data[data['Age'] >= 18]
adults_survival_rate = adults['Survived'].mean()
print(adults_survival_rate)
```

7. Get the median age of survivors

```
median_age_survivors = data[data['Survived'] == 1]['Age'].median()
print(median_age_survivors)
```

8. Get the median age of non-survivors

```
median_age_non_survivors = data[data['Survived'] == 0]['Age'].median()
print(median_age_non_survivors)
```

9. Get the median fare of survivors

```
median_fare_survivors = data[data['Survived'] == 1]['Fare'].median()
print(median_fare_survivors)
```

10. Get the median fare of non-survivors

```
median_fare_non_survivors = data[data['Survived'] == 0]['Fare'].median()
print(median_fare_non_survivors)
```

